

Design Video Streaming

YouTube · Netflix · Hotstar – "video ఎక్స్కించి, transcode చేసి, CDN lo పెడదాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – ఒక 2-గంటల video **5.6 రోజులు** పడుతుంది, మరియు **workers ని 128 రెట్లు పెంచినా** అది ఒక్క నిమిషం కూడా తగ్గదు. రెండోది – మీరు transcode చేసిన వాటిలో **45.3% videos ఎవరూ చూడరు**. మూడోది – అదే 6 TB cache మీ content ని బట్టి **25.2% లేదా 98.9%** hit rate ఇస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

ఈ PROBLEM LO మిగతా అన్నిటికంటే వేరైన విషయం

ఇప్పటివరకు చూసిన systems lo పని ముక్కలు చిన్నవి: ఒక request, ఒక message, ఒక notification. అవి మిల్లీసెకన్ల పనులు, మరియు వాటిని పంచడం సులభం.

ఇక్కడ ఒక్క పని ముక్క **5.6 రోజులు** పడుతుంది.

అదే ఈ problem యొక్క కేంద్రం, మరియు §4 lo దాన్ని కొలుస్తాం: **workers ని 16 నుంచి 2,048 కి పెంచినా వాస్తవ ఆలస్యం ఒక్క సెకను కూడా తగ్గదు.** ఎందుకంటే మీ pool ఎంత పెద్దదైనా, ఒక పనిని రెండు machines మీద నడపలేరు.

సమాంతరత మీ workers సంఖ్య మీద కాదు – మీ అతి పెద్ద అవిభాజ్య ముక్క మీద ఆధారపడుతుంది.

మరియు §7, §10 రెండూ ఒకే ఆకారంలో ఉంటాయి: **మీరు చేస్తున్న పనిలో ఎంత నిజంగా అవసరం?**

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – ఒక video, ఇరవై నాలుగు పైళ్ళు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: పంచలేని పని

3. Step – ప్రతి rendition ఒక job
4. మొదటి విరుపు – 2,048 workers, అదే 5.6 రోజులు
5. Step – video ని ముక్కలుగా చేయడం

Part 3 – రెండో విరుపు: ఎవరూ చూడని videos

6. Step – upload అవగానే అన్ని రూపాలు
7. రెండో విరుపు – 45.3% videos ఎవరూ చూడరు

Part 4 – మూడో విరుపు: CDN

8. Step – edge lo cache చేయడం
9. మూడో విరుపు – అదే cache, 25% లేదా 99%

Part 5 – పూర్తి system

10. మొత్తం code · 47 లక్షల segment jobs · mutation testing

Part 6 – Interview lo

11. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
12. నోటితో చెప్పాల్సిన English script
13. Follow-ups – live streaming, DRM, thumbnails
14. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – ఒక video, ఇరవై నాలుగు పైళ్ళు

ఒకరు video ఎక్కిస్తారు. మిగతా అందరూ దాన్ని చూస్తారు – ఫోన్ lo 2G మీద, TV lo fibre మీద, laptop lo WiFi మీద.

ఆ "అందరి పరికరాల్లో" అనేదే ఈ problem. ఒక్క పైలు అందరికీ సరిపోదు:

ఒక video → ఎన్ని outputs?

```
renditions : 8
codecs      : 3
మొత్తం      : 8 × 3 = 24 outputs
```

- **8 renditions** – 144p నుంచి 4K వరకు, ఎందుకంటే 2G నెట్వర్క్ 4K మోయలేదు
- **3 codecs** – H.264 (అన్ని పరికరాల్లో పని చేస్తుంది), VP9 (30% చిన్నది), AV1 (50% చిన్నది, కానీ పాత పరికరాల్లో లేదు)

కాబట్టి **ఒక upload = 24 పైళ్ళు**. మరియు ఆ 24 ఒకేలా ఖరీదు కావు – 4K/AV1 ఒక్కటే 144p/H.264 కంటే **వేల రెట్లు** CPU తింటుంది.

ఈ **PROBLEM** యొక్క ఆకారం – మిగతా వాటికంటే ఎందుకు వేరు

Chat lo ఒక message **మిల్లీసెకన్లు**. Feed lo ఒక fan-out **మిల్లీసెకన్లు**.

ఇక్కడ ఒక పని ముక్క **రేజులు** పడుతుంది, మరియు అది **CPU-bound** – network కోసం ఎదురుచూడటం కాదు, నిజంగా లెక్కించడం.

అంటే LLD Deep 26 §12 lo చూసిన సూత్రం ($\text{cores} \times (1 + \text{ఎదురుచూపు/CPU})$) ఇక్కడ **ఎదురుచూపు = 0** ఇస్తుంది – అంటే **threads = cores**. మీరు threads పెంచి ఏమీ సాధించలేరు; మీకు కావాల్సినవి **machines**.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Upload అయిన ఎంతసేపటికీ video కనిపించాలి?	ఇదే §4 మొత్తం. "వెంటనే" అంటే ముక్కలు తప్పనిసరి
ఎన్ని renditions, ఎన్ని codecs తప్పనిసరి?	ప్రతి codec మీ transcode ఖర్చుని గుణిస్తుంది
Videos ఎంత పొడవు?	3 నిమిషాలా, 3 గంటలా – §4 lo అది 5.6 రేజుల తేడా
ఎన్ని శాతం videos నిజంగా చూడబడతాయి?	45.3% ఎప్పుడూ చూడబడవు – §7
Live streaming ఉందా?	ఉంటే ఇది పూర్తిగా వేరే problem – §13
CDN ఎంత పెద్దది పెట్టుకోగలం?	§9 – మరియు ఆ జవాబు మీ content మీద ఆధారపడుతుంది, cache మీద కాదు
ప్రజాదరణ ఎంత కేంద్రీకృతం?	§9 – ఇదే మీ CDN hit rate ని 25% లేదా 99% చేస్తుంది

ఒక ప్రశ్న అడగడం మర్చిపోతే మీ మొత్తం అంచనా తప్పు అవుతుంది

"Upload అయిన తర్వాత video ఎంతసేపటికీ చూడగలగాలి?"

"కొన్ని నిమిషాల్లో" అంటే – §5 యొక్క ముక్కలు తప్పనిసరి, మరియు మీకు వేల machines కావాలి.

"కొన్ని గంటలు ఫరవాలేదు" అంటే – చాలా సులభమైన pipeline సరిపోతుంది.

ఆ ఒక్క వాక్యం మీ architecture ని పూర్తిగా మారుస్తుంది, మరియు చాలామంది దాన్ని అడగరు.

Part 2 – మొదటి విరుపు: పంచలేని పని

3. Step – ప్రతి rendition ఒక job

సహజమైన pipeline:

```
function onUpload(video) {  
  for (const rendition of RENDITIONS) {           // 8  
    for (const codec of CODECS) {                 // 3  
      queue.push({ video, rendition, codec }); // = 24 jobs  
    }  
  }  
}
```

24 jobs, ఒక worker pool. Workers తక్కువైతే machines పెంచుతాం. ఇది **సరైన ఆలోచన** లాగా ఉంది – మనం ఇప్పటివరకు చూసిన ప్రతి system lo అది పని చేసింది.

కొలిచి చూద్దాం.

4. మొదటి విరుపు – 2,048 workers, అదే 5.6 రోజులు

ఒక 2-గంటల video • 24 outputs • వేర్వేరు workers

విభజన		పనుల సంఖ్య		అతి పెద్ద పని		16 workers		256 workers		2048 workers
విభజించకుండా		24		5.6 రోజులు		5.6 రోజులు		5.6 రోజులు		5.6 రోజులు

WORKERS ని 128 రెట్లు పెంచాం. ఆలస్యం ఒక్క సెకను కూడా తగ్గలేదు.

కారణం "అతి పెద్ద పని" నిలువు వరుసలో ఉంది: **5.6 రోజులు**.

అది ఒక్క job – 4K/AV1. మీ దగ్గర 2,048 machines ఉన్నా, **ఒక job ని రెండు machines మీద నడపలేరు**. మిగతా 2,047 ఖాళీగా కూర్చుని ఆ ఒక్కదాని కోసం ఎదురుచూస్తాయి.

ఇది queue నిండటం కాదు, workers తక్కువ కావడం కాదు. **ఇది గణితం**.

4K/AV1 – 5.6 రేజులు · ఒక్క machine మీద

4K/VP9 – 1.7 రేజులు

1440p/AV1

1080p/AV1 ... మిగతా 21 పనులు

ఇక్కడ ముగుస్తుంది

10 సెకన్ల ముక్కలుగా విభజిస్తే – 17,280 చిన్న పనులు

అతి పెద్ద పని: **11.1 నిమిషాలు** · 2,048 workers తో మొత్తం **11.1 నిమిషాలు**

మొత్తం CPU పని రెండింటిలోనూ ఒకటే – 14 రేజులు.

మారినది పని మొత్తం కాదు; మారినది అతి పెద్ద ముక్క

5. Step – video ని ముక్కలుగా చేయడం

Video కి ఒక ప్రత్యేక గుణం ఉంది, మరియు అదే ఈ problem కి తాళం: **దాన్ని ముక్కలుగా కోసి, ఒక్కో ముక్కని స్వతంత్రంగా transcode చేసి, తిరిగి అతికించొచ్చు.**

(ఇది GOP సరిహద్దుల దగ్గర కోయాలి – లేకపోతే ముక్కలు స్వతంత్రంగా decode కావు. అదే HLS/DASH చేసేది.)

```
// ఒక rendition కోరడం = దాని ప్రతి ముక్కకీ ఒక job
#request(videoId, rend) {
  const v = this.#videos.get(videoId);
  if (!v || v.renditions.has(rend)) return false;
  v.renditions.set(rend, { done: new Set(), total: v.segments });
  for (let i = 0; i < v.segments; i++) {
    this.#queue.push({ id: this.#nextJob++, videoId, rend, seg: i });
    this.stats.jobsCreated++;
  }
  return true;
}
```

విభజన	పనుల సంఖ్య	అతి పెద్ద పని	16 workers	256 workers	2048 workers
విభజించకుండా	24	5.6 రేజులు	5.6 రేజులు	5.6 రేజులు	5.6 రేజులు
60 సెకన్ల ముక్కలు	2,880	1.1 గం	20.9 గం	1.3 గం	1.1 గం
10 సెకన్ల ముక్కలు	17,280	11.1 ని	20.9 గం	1.3 గం	11.1 ని

(మొత్తం CPU పని మూడు వరుసల్లోనూ ఒకటే – 14.0 రేజులు)

5.6 రోజులు → 11.1 నిమిషాలు · 730 రెట్లు · మరియు CPU పని ఒక్క సెకను కూడా తగ్గలేదు

ఆ చివరి వాక్యం ముఖ్యం. మనం **వేగవంతం చేయలేదు** – మనం **పంచగలిగేలా** చేశాం.

మరియు ఆ పట్టికలో ఇంకొక పాఠం ఉంది: **10 సెకన్ల ముక్కలు 16 workers తో 20.9 గంటలు** – 60 సెకన్ల ముక్కల మాదిరిగానే. ఎందుకంటే 16 workers దగ్గర **workers ఇక అడ్డంకి**, ముక్కల పరిమాణం కాదు.

అంటే ముక్కలు చేయడం మరియు workers పెంచడం – **రెండూ కలిసి** పని చేస్తాయి, విడిగా కాదు.

ఒక ఉచిత లాభం – ఇదే ADAPTIVE BITRATE ని సాధ్యం చేస్తుంది

ముక్కలు చేయడం ఒక transcode తంత్రం మాత్రమే కాదు. అవే ముక్కలు **ప్లేయర్ కూడా** వాడుతుంది: ప్రతి 10 సెకన్లకీ నెట్‌వర్క్ వేగాన్ని చూసి **వేరే rendition కి మారొచ్చు**.

కాబట్టి §5 యొక్క నిర్ణయం రెండు వేర్వేరు సమస్యలని ఒకేసారి పరిష్కరిస్తుంది – **transcode సమాంతరత** మరియు **adaptive bitrate**. అందుకే HLS/DASH రెండూ ఇలాగే ఉంటాయి.

Part 3 – రెండో విరుపు: ఎవరూ చూడని videos

6. Step – upload అవగానే అన్ని రూపాలు

ఇప్పుడు pipeline వేగంగా ఉంది. కాబట్టి సహజమైన తర్వాతి అడుగు: ప్రతి upload కీ అన్ని 24 రూపాలూ వెంటనే తయారు చేయడం.

అప్పుడు ఎవరు ఎప్పుడు చూసినా అది సిద్ధంగా ఉంటుంది. మంచి అనుభవం.

కానీ ఒక ప్రశ్న: అసలు ఎంతమంది చూస్తారు?

7. రెండో విరుపు – 45.3% videos ఎవరూ చూడరు

500k videos · views పంపిణీ

p50 : 1 p90 : 13 p99 : 196 గరిష్ఠం : 95,71,795

ఎప్పుడూ చూడనివి : 2,26,296 (45.3%)

10 కంటే తక్కువ : 4,38,012 (87.6%)

ఫైన 1% videos → మొత్తం views lo 89.8%

మీ TRANSCODE ఖర్చులో సగం ఎవరూ చూడని VIDEOS మీద

45.3% videos ఒక్క view కూడా పొందవు. **87.6%** కి పది కంటే తక్కువ.

మరియు ఫైన 1% videos మొత్తం views lo **89.8%** తీసుకుంటాయి.

అంటే మీరు 4K/AV1 transcode చేస్తున్న ప్రతి వంద videos lo ఒక్కటి మాత్రమే ఆ నాణ్యత అవసరమయ్యేంత మంది చూస్తారు.

పరిష్కారం: ముందే కొన్ని, మిగతావి డిమాండ్ మీద.

```
const EAGER = ['360p/H.264', '720p/H.264']; // §7 – ముందే ఇవి మాత్రమే
```

మిగతా 22 రూపాలు ఎవరైనా నిజంగా అడిగినప్పుడే:

```
// §7 – చూసినప్పుడే మిగతా రూపాలు తయారు చేయడం
watch(videoId, wanted) {
```

```

...
if (this.ready(videoId, wanted)) { this.stats.served++; return { ok: true, rend:
wanted }; }
// ఇంకా లేదు → తయారు చేయమని అడిగి, ఇప్పటికీ సిద్ధమైన దాన్ని ఇవ్వు
if (this.#request(videoId, wanted)) this.stats.lazyTriggered++;
const have = this.manifest(videoId);
if (!have.length) return { ok: false, reason: 'NOT_READY' };
this.stats.servedFallback++; this.stats.served++;
return { ok: true, rend: have[have.length - 1], fallback: true };
}

```

ఆ చివరి పంక్తి ముఖ్యం: **వాడుకరి ఎదురుచూడడు**. అతనికి ఇప్పుడు సిద్ధంగా ఉన్న అత్యుత్తమ నాణ్యత వెంటనే ఇస్తాం, మరియు అతను కోరినది **తర్వాతి సారికి** తయారవుతుంది.

ఒక సగటు video (12 ని) ని 24 రూపాల్లోకి మార్చడానికి ≈ 34 CPU గంటలు

వ్యాసం	transcode చేసినవి	CPU గంటలు	వృథా
అన్నిటిని ముందే, అన్ని రూపాల్లో	5,00,000	16.8 మి	–
3 రూపాలు ముందే • మిగతావి డిమాండ్ మీద	1,209	2.1 మి	87%

87% CPU ఆదా – మరియు వాడుకరికి తేడా దాదాపు తెలియదు

ఎందుకంటే 87.6% videos కి పది కంటే తక్కువ views. వాటిని ఎవరైనా చూస్తే 720p సరిపోతుంది.

మరియు ప్రజాదరణ పొందిన 1% – వాటికి మొదటి కొన్ని వందల views తర్వాత అన్ని రూపాలూ సిద్ధంగా ఉంటాయి.

ఇది HLD Deep 03 §8 మరియు HLD Deep 05 §7 lo చూసిన అదే ప్రశ్న: **"నేను చేస్తున్న ఈ పనిని ఎవరైనా నిజంగా వాడతారా?"** – మరియు మళ్ళీ జవాబు "చాలావరకు లేదు".

Part 4 – మూడో విరుపు: CDN

8. Step – edge lo cache చేయడం

Transcode అయిన పైళ్ళు పెద్దవి. వాటిని ప్రతిసారీ origin నుంచి పంపితే – bandwidth ఖర్చు, మరియు దూరంగా ఉన్నవాళ్ళకి నెమ్మది.

కాబట్టి **CDN**: ప్రపంచమంతా edge servers, ప్రతిదానిలో ఒక cache.

ప్రశ్న సూటిగా ఉంది: **ఆ cache ఎంత పెద్దది ఉండాలి?**

9. మూడో విరుపు – అదే cache, 25% లేదా 99%

200k videos · ఒక్కటి ~0.6 GB · మొత్తం 120 TB
20 లక్షల requests · Zipf ప్రజాదరణ

edge cache	ఎన్ని videos పడతాయి	మొత్తంలో %	cache hit rate	origin కి వెళ్ళేవి
60 GB	100	0.1%	13.5%	86.5%
300 GB	500	0.3%	24.5%	75.5%
1.5 TB	2,500	1.3%	38.2%	61.8%
6 TB	10,000	5.0%	52.8%	47.2%
30 TB	50,000	25.0%	74.3%	25.7%

ఇది ఉపయోగకరంగా అనిపిస్తుంది – 5% cache చేస్తే 52.8% hits. కానీ ఆగండి.

ఈ మొత్తం పట్టిక **ఒక ఊహ** మీద ఆధారపడి ఉంది: ప్రజాదరణ ఎంత కేంద్రీకృతమైంది. దాన్ని మార్చి చూద్దాం:

ఇది ప్రజాదరణ ఎంత కేంద్రీకృతమైందో దాని మీద ఆధారపడుతుంది –
కాబట్టి ఆ ఊహని విడిగా చూపిస్తున్నాను (6 TB cache):

Zipf ఘాతం	అర్థం	cache hit rate
0.7	చదువైన తోక · niche ఎక్కువ	25.2%
0.9	మధ్యస్థం	52.8%
1.2	కొన్నే hits	90.0%
1.5	చాలా కేంద్రీకృతం	98.9%

ఒకే CACHE · ఒకే పరిమాణం · 25.2% నుంచి 98.9%

Cache technology మారలేదు. Cache పరిమాణం మారలేదు. LRU విధానం మారలేదు.

మరింది **మీ content యొక్క స్వభావం** – కొన్ని పెద్ద hits ఉన్నాయా, లేక చాలా niche videos ఉన్నాయా.

మీ CDN sizing ఒక engineering నిర్ణయం కాదు – అది మీ catalogue యొక్క ఒక లక్షణం.

అందుకే §2 lo "ప్రజాదరణ ఎంత కేంద్రీకృతం?" అని అడగాలి. ఆ జవాబు తెలియకుండా మీరు ఇచ్చే ఏ CDN అంచనా అయినా **నాలుగు రెట్లు** తప్పు కావచ్చు.

NETFLIX మరియు YOUTUBE ఒకే PROBLEM కాదు

Netflix – కొన్ని వేల titles, అందరూ అవే చూస్తారు. ఘాతం ఎక్కువ → **cache దాదాపు అంతా పట్టుకుంటుంది.** అందుకే Netflix ISP lo సొంత పెట్టెలు పెట్టగలదు.

YouTube – కోట్లాది videos, చాలావరకు niche. ఘాతం తక్కువ → **cache ఎప్పటికీ సరిపోదు.** అందుకే YouTube కి బహుళ-అంచెల cache మరియు భారీ origin కావాలి.

ఒకే architecture రెండింటికీ సరిపోదు, మరియు ఆ తేడా code lo కాదు – అది content lo ఉంది.

Part 5 – పూర్తి system

10. మొత్తం code · 47 లక్షల segment jobs · mutation testing

నాలుగు నియమాలు

#	నియమం	విరిగితే అర్థం
1	ఇచ్చిన rendition యొక్క అన్ని ముక్కలూ పూర్తయి ఉండాలి	సగం transcode అయినది ఇస్తున్నాం
2	manifest lo ఉన్న ప్రతిదీ పూర్తిగా సిద్ధం	ప్లేయర్ లేని పైలు అడుగుతుంది
3	పూర్తయిన ముక్కలు మొత్తం ముక్కలని దాటకూడదు	లెక్క విరిగింది
4	ప్రతి upload కి eager renditions కోరబడాలి	కొన్ని videos ఎప్పటికీ చూడలేం

నియమం 1 ఈ problem కి ప్రత్యేకమైనది: ఒక rendition "కొంచెం సిద్ధం" అనేది ఉండదు. 17,280 ముక్కల్లో 17,279 అయినా అది ఇంకా సిద్ధం కాదు.

1,000 యాదృచ్ఛిక ప్రయోగాలు · 37,164 uploads · 46,81,173 segment jobs
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

uploaded	37,164
jobsCreated	46,81,173
jobsDone	2,77,677
jobsDup	41,764
lazyTriggered	1,03,742
servedFallback	11,868
served	18,849

lazyTriggered **1,03,742** – అంటే §7 యొక్క డిమాండ్-ఆధారిత transcode నిజంగా నడుస్తోంది.

servedFallback **11,868** – వాడుకరి కోరిన నాణ్యత ఇంకా సిద్ధం కానప్పుడు తక్కువ నాణ్యత ఇచ్చాం, మరియు ఎవరూ ఎదురుచూడలేదు.

Test విఫలం కాగలదా? – నాలుగు మార్పులు

మార్పు లేని code → 0/1000 విఫలం

"అన్ని ముక్కలు అయ్యాయా" పరీక్ష తీసేస్తే → 300/300 విఫలం

ఉదా: v15: అసంపూర్ణ 360p/H.264 ఇచ్చాం (1/28)

manifest lo సిద్ధం కానివి కూడా చూపిస్తే → 300/300 విఫలం

ఉదా: v15: అసంపూర్ణ 720p/H.264 ఇచ్చాం (0/28)

ఒకే ముక్క రెండుసార్లు లెక్కితే → 0/300 విఫలం

మొదటి మార్పు ముందు పట్టుబడలేదు – మరియు కారణం నా TEST LO ఉంది

నా నియమం 1 ఇలా ఉండేది: `if (!p.ready(id, res.rend)) bad.push(...)`.

కానీ `ready()` అనేది **నేను పరీక్షిస్తున్న code యొక్క భాగమే**. దాన్ని మార్చినప్పుడు, code మరియు పరీక్ష **రెండూ కలిసి** మారాయి – కాబట్టి పరీక్ష ఎప్పుడూ pass అయింది. **0/300**.

దాన్ని సరిచేయడం: ముక్కల లెక్కని **స్వతంత్రంగా** చూడటం –

```
const st = p.video(id).renditions.get(res.rend);  
if (st.done.size !== st.total) bad.push(...)
```

అప్పుడు అదే మార్పు **300/300** పట్టుబడింది, మరియు సందేశం **"1/28 ముక్కలు"** అని స్పష్టంగా చెప్పింది.

ఒక నియమం, అది పరీక్షిస్తున్న code ని వాడితే, అది ఒక నియమం కాదు – అది ఒక ప్రతిభ్యసి.

మూడో మార్పు (ఒకే ముక్క రెండుసార్లు) **0/300** – మరియు దానికి కారణం నిజాయితీగా చెప్పాలి: `done` ఒక `Set`, కాబట్టి అదే ముక్కని రెండోసారి చేర్చడం **ఎలాగూ ప్రభావం చూపదు**. ఆ `if` లెక్కల కోసం మాత్రమే, సరి-తప్పు కోసం కాదు. **అది దాదాపు చచ్చిన code**, మరియు test సరిగ్గానే అది చెప్పింది.

Part 6 – Interview Io

11. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి
0-5	8 × 3 = 24 outputs బొద్దు మీద. "ఒక upload = 24 పైళ్ళు"
5-9	§2 ప్రశ్నలు. ముఖ్యంగా "ఎంతసేపటికీ కనిపించాలి?" మరియు "ప్రజాదరణ ఎంత కేంద్రీకృతం?"
9-17	మొదటి విరుపు. 24 jobs → 2,048 workers , అదే 5.6 రోజులు . "అతి పెద్ద అవిభాజ్య ముక్క"
17-25	ముక్కలు → 11.1 నిమిషాలు , CPU పని అదే. మరియు అవే ముక్కలు ABR కి కూడా
25-33	రెండో విరుపు. 45.3% ఎవరూ చూడరు. Eager 2 + lazy → 87% ఆదా , fallback తో ఎదురుచూపు లేకుండా
33-40	మూడో విరుపు. CDN – 25.2% vs 98.9% , content ని బట్టి. Netflix vs YouTube
40-45	నాలుగు నియమాలు, మరియు "నా test తనను తానే పరీక్షించుకుంటోంది"

ఏమి తప్పక చెప్పాలి

- సమాంతరత **workers** మీద కాదు, అతి పెద్ద అవిభాజ్య ముక్క మీద ఆధారపడుతుంది.
- ముక్కలు చేయడం **transcode** ని వేగవంతం చేయదు – అది దాన్ని పంచగలిగేలా చేస్తుంది. CPU పని అదే.
- అవే ముక్కలు adaptive bitrate** ని సాధ్యం చేస్తాయి. ఒక నిర్ణయం, రెండు పరిష్కారాలు.
- 45.3% videos** ఎవరూ చూడరు – కాబట్టి eager transcode ని కనిష్టంగా ఉంచాలి.
- CDN hit rate ఒక content లక్షణం, cache లక్షణం కాదు.** 25% లేదా 99%.

ఏమి వదిలేయాలి

- Codec వివరాలు (GOP, B-frames, keyframe interval) – ఒక వాక్యం
- Upload resumability, chunked upload – అడిగితే మాత్రమే
- Player buffer algorithm – §13 lo ఒక వాక్యం
- Thumbnail generation – ఇది transcode pipeline యొక్క చిన్న అనుబంధం

12. నీటితో చెప్పాల్సిన English script

"One upload isn't one file – it's **eight renditions times three codecs, twenty-four outputs**, and they're wildly unequal: 4K/AV1 alone costs thousands of times more CPU than 144p/H.264. That asymmetry is where this problem lives."

"The obvious pipeline queues twenty-four jobs and adds machines when it's slow. I measured a two-hour video: **sixteen workers take 5.6 days, 256 workers take 5.6 days, 2,048 workers take 5.6 days**. Not a queueing problem, not a capacity problem – the single 4K/AV1 job *is* 5.6 days, and you can't run one job on two machines. **Parallelism is bounded by your largest indivisible unit, not your worker count.**"

"Video has a property that saves us: you can cut it at GOP boundaries, transcode the pieces independently, and concatenate. With ten-second segments that's 17,280 small jobs instead of 24 huge ones, and the longest is **11 minutes**. Wall time goes from 5.6 days to 11 minutes – **730x** – and I'd stress that **the total CPU work is identical, 14 days either way**. We didn't make it faster, we made it divisible. There's also a free win: those same segments are what lets the player switch quality every ten seconds, so segmentation solves transcoding parallelism and adaptive bitrate with one decision. That's why HLS and DASH look the way they do."

"Second: now that the pipeline is fast, should we transcode everything? I measured the view distribution. **45.3% of videos are never watched at all**, 87.6% get under ten views, and the top 1% take **89.8% of all views**. So eagerly producing 4K/AV1 for every upload means doing the most expensive work for content nobody will see. I'd transcode two renditions eagerly – 360p and 720p H.264 – and produce the rest **on first request**. That's an **87% CPU saving**, and the viewer never waits: if they ask for a rendition that isn't ready, I serve the best one that *is* and queue theirs for next time."

"Third, the CDN, and this is the one I'd be most careful about. A 6 TB edge cache holding 5% of the catalogue gets a 52.8% hit rate. But that number is almost entirely an assumption about **how concentrated popularity is**. I swept it: the **same** 6 TB cache gives **25.2%** if the tail is flat and **98.9%** if a few titles dominate. Same hardware, same LRU, four times the difference. **CDN sizing is a property of your catalogue, not of your cache**. It's why Netflix – a few thousand titles everyone watches – can ship boxes into ISPs, and YouTube can't."

"Four invariants, a thousand runs, 4.7 million segment jobs, zero violations. The one worth calling out is that a rendition is never 'partly ready' – 17,279 of 17,280 segments is still not ready. And a testing lesson: that invariant originally checked completeness by calling the system's own `ready()` method, so when I mutated `ready()`, the code and the check moved together and the mutation passed **0 of 300**. Rewriting it to count segments independently caught the same mutation **300 of 300**. **An invariant that calls the code it's testing isn't an invariant, it's an echo.**"

13. Follow-ups – live streaming, DRM, thumbnails

"Live streaming ఉంటే ఏమి మారుతుంది?"

దాదాపు అంతా. §5 యొక్క ముక్కలు ఇంకా వాడతాం, కానీ ఇప్పుడు అవి **నిజ సమయంలో** రావాలి – అంటే transcode **1x వేగం కంటే వేగంగా** ఉండాలి, లేకపోతే మీరు వెనకబడిపోతారు. అది AV1 మరియు 4K ని చాలా సందర్భాల్లో **అసాధ్యం** చేస్తుంది (§1 పట్టికలో 4K/AV1 వేగం 0.15x మాత్రమే). కాబట్టి live lo renditions తక్కువ, codecs తక్కువ, మరియు **latency ఒక మొదటి-తరగతి constraint** (LL-HLS తో 2-5 సెకన్లు). మరియు §7 lo చూసిన lazy transcode **పూర్తిగా పనికిరాదు** – live lo "తర్వాతి సారికి తయారు చేద్దాం" అనేది ఉండదు.

"DRM ఎక్కడ కూర్చుంటుంది?"

Transcode తర్వాత, packaging దశలో. ప్రతి ముక్కని encrypt చేసి, keys ని ఒక license server ఇస్తుంది. ముఖ్యమైన పర్యవసానం: **encrypt అయిన ముక్కలని CDN cache చేయగలదు** (అవి అందరికీ ఒకటే), కానీ **keys cache కాకూడదు**. ఆ విభజన లేకపోతే మీ CDN hit rate సున్నా అవుతుంది.

"Thumbnails ఎలా?"

అదే segment pipeline, కానీ చాలా చౌక – ప్రతి n సెకన్లకీ ఒక frame. ఇక్కడ ఒక ఆచరణాత్మక విషయం: **thumbnails eager గా చేయాలి**, renditions కాకపోయినా. ఎందుకంటే thumbnail లేని video **ఎవరూ click చేయరు** – అంటే §7 యొక్క "డిమాండ్ మీద" ఎప్పటికీ trigger కాదు. **Lazy transcode ఒక చక్రంలో చిక్కుకోకూడదు: చూస్తేనే తయారవుతుంది, కానీ తయారైతేనే చూస్తారు.**

"Origin storage ఎంత?"

24 రూపాలు × అన్ని videos అంటే అది భారీ. కానీ §7 తర్వాత చాలా videos కి **2 రూపాలే** ఉంటాయి. మరియు ఒక అదనపు ఆదా: **చాలా కాలం ఎవరూ చూడని videos యొక్క ఖరీదైన రూపాలని తీసేయొచ్చు** – అవి మళ్ళీ అవసరమైతే తిరిగి తయారు చేసుకోవచ్చు. Transcode ఒకసారి చేసిన పని కాదు, అది ఒక **పునరుత్పత్తి చేయగల పని** – అదే దీన్ని cache లాగా చూడనిస్తుంది.

"ఒక ముక్క transcode విఫలమైతే?"

అది మిగతా 17,279 ముక్కలని ప్రభావితం చేయకూడదు – దాన్ని మాత్రమే మళ్ళీ ప్రయత్నించాలి. ఇది ముక్కలు చేయడం యొక్క ఇంకొక లాభం: **వైఫల్యం కూడా ముక్కలవుతుంది**. అవిభాజ్య job lo 5.6 రోజుల చివరిలో విఫలమైతే, మీరు 5.6 రోజులు పోగొట్టుకుంటారు.

14. ఏమి నేర్చుకున్నాం

1. సమాంతరత workers మీద కాదు, అతి పెద్ద అవిభాజ్య ముక్క మీద ఆధారపడుతుంది. 16 నుంచి 2,048 workers – అదే 5.6 రోజులు. ఒక job ని రెండు machines మీద నడపలేరు.
2. ముక్కలు చేయడం పనిని తగ్గించదు, దాన్ని పంచగలిగేలా చేస్తుంది. మొత్తం CPU పని రెండింటిలోనూ 14 రోజులు – కానీ వాస్తవ ఆలస్యం 5.6 రోజుల నుంచి 11.1 నిమిషాలు.
3. ఒక మంచి విభజన ఒకటి కంటే ఎక్కువ సమస్యలని పరిష్కరిస్తుంది. అవే 10-సెకన్ల ముక్కలు transcode సమాంతరత, adaptive bitrate, మరియు వైఫల్య పునరుద్ధరణ – మూడింటినీ ఇస్తాయి.
4. మే ఖరీదైన పనిలో ఎంత నిజంగా అవసరం? 45.3% videos ఎవరూ చూడరు. Eager renditions ని రెండింటికీ తగ్గించడం 87% CPU ఆదా చేసింది.
5. వాడుకరిని ఎదురుచూపించకుండా lazy చేయొచ్చు. అతను కోరిన నాణ్యత లేకపోతే, ఉన్నదాంట్లో ఉత్తమమైనది ఇచ్చి కోరినదాన్ని తర్వాత తయారు చేయడం.
6. CDN hit rate ఒక content లక్షణం, cache లక్షణం కాదు. ఒకే 6 TB – 25.2% లేదా 98.9%. మే catalogue ఆకారం తెలియకుండా CDN అంచనా వేయడం అర్థరహితం.
7. తనను తానే పరీక్షించుకునే నియమం ఒక నియమం కాదు. నా పూర్ణత నియమం ready() ని పిలిచింది – మరియు ready() ని మార్చినప్పుడు నియమం కూడా మారింది, కాబట్టి 0/300. స్వతంత్రంగా లెక్కపెట్టాక 300/300.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఈ problem lo మే అతి పెద్ద శత్రువు పని మొత్తం కాదు – ఆ పని యొక్క ఆకారం.

14 రోజుల CPU పని రెండు రూపాల్లో ఉండొచ్చు: 24 భారీ ముక్కలు (5.6 రోజులు) లేదా 17,280 చిన్న ముక్కలు (11 నిమిషాలు).

అదే పని. అదే machines. 730 రెట్లు తేడా – కేవలం ఆకారం వల్ల.